

Experiment No.	9
Experiment Title	Perceptron Learning Algorithm vs. Multi-Layer Perceptron for Handwritten English Character Recognition
GitHub Repository	https://github.com/Dharika-2006/ML-LAB-ASSIGNMENTS
Student Name	Dharika Shree K
Register Number	3122247001014
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	<i>fill in</i>

1 Objective

To implement the Perceptron Learning Algorithm (PLA) from scratch and a Multi-Layer Perceptron (MLP) trained by back-propagation, and to compare the two on a 62-class handwritten English character recognition task. The MLP is tuned over a hyper-parameter space covering network architecture (width and depth), activation function (ReLU, sigmoid, tanh), loss function (cross-entropy, squared error), optimizer (full-batch gradient descent, mini-batch SGD, SGD with momentum, Adam), learning rate, batch size, dropout and L2 weight decay, using a two-part strategy of random search followed by greedy stage-wise (coordinate) search. The experiment aims to establish why a single-layer perceptron underperforms on a non-linearly-separable problem, which hyper-parameters dominate MLP performance, how optimizer choice affects convergence, whether depth always helps, and whether the tuned MLP overfits.

2 Problem Statement

Problem: Classify grayscale images of handwritten English characters into 62 classes (digits 0–9, uppercase A–Z, lowercase a–z) using (A) a one-vs-rest Perceptron Learning Algorithm with step activation, and (B) a tuned Multi-Layer Perceptron, and determine how much of the performance gap is attributable to non-linear feature learning.

Input: The English Handwritten Characters dataset (Kaggle, [dhruvildave/english-handwritten](https://www.kaggle.com/dhruvildave/english-handwritten)) containing 3410 labelled character images together with an `english.csv` index of image paths and labels. Each image is converted to grayscale, resized to 32×32 , inverted so that ink is 1 and background is 0, scaled to $[0, 1]$, flattened to a 1024-dimensional vector, and standardized using training-split statistics.

Output: A trained PLA model and a tuned MLP; per-epoch convergence curves for both; random-search and stage-wise tuning tables; a hyper-parameter impact ranking; an optimizer $\times$ learning-rate grid; test-set accuracy, macro/weighted precision, recall and F1, micro/macro one-vs-rest ROC-AUC and confusion matrices for both models; a regularised vs. unregularised overfitting comparison; and a final A/B comparison of PLA against the tuned MLP.

3 Dataset Description

Dataset Name	English Handwritten Characters Dataset
--------------	--

Dataset Source	Kaggle — <code>dhruvildave/english-handwritten-characters-d</code> (v3), downloaded via <code>kagglehub</code>
Number of Samples	3410 images
Number of Features	1024 (32×32 grayscale pixels, flattened)
Number of Classes	62 (0–9, A–Z, a–z); approximately 55 samples per class
Missing Values	None — every row of <code>english.csv</code> resolves to a readable image file
Train/Val/Test Split	70% train (2387) / 15% validation (511) / 15% test (512), stratified, <code>random_state=42</code>

4 Exploratory Data Analysis

- Dataset overview: 3410 images, 62 classes, roughly balanced at ≈ 55 images per class.
- Per-class sample counts (near-uniform, so accuracy and macro-F1 are both meaningful here, unlike an imbalanced dataset).

Class Balance

With 3410 images spread over 62 classes, the dataset is close to uniform (mean ≈ 55 images/class). Stratified splitting was nonetheless used for both the train/temp split and the validation/test split, so that every class is represented in all three partitions. Because the classes are near-balanced, plain accuracy is not misleading here; macro-averaged precision, recall and F1 were still reported alongside it so that rare-confusion classes are not hidden by the majority behaviour.

5 Data Preprocessing

- **Image loading:** Each path listed in `english.csv` is opened with PIL and converted to single-channel grayscale (`convert("L")`).
- **Resizing:** All images are resized to 32×32 with bilinear interpolation, giving a fixed 1024-dimensional input vector.
- **Intensity normalisation and inversion:** Pixel values are divided by 255 and then inverted ($1 - x$), so that ink pixels are close to 1 and background close to 0. This makes the representation sparse in the informative direction, which suits both the perceptron update and ReLU units.
- **Flattening:** Each image is raveled to a 1024-vector; no convolutional structure is used, so all spatial information must be recovered by the fully-connected layers.
- **Label encoding:** `LabelEncoder` maps the 62 character labels to integers 0..61.
- **Train/validation/test split:** A stratified 70/30 split is followed by a stratified 50/50 split of the held-out portion, producing 2387 training, 511 validation and 512 test images (`random_state=42`).

- **Standardization:** Per-feature mean and standard deviation are computed *on the training split only* ($\sigma + 10^{-6}$ for numerical safety) and applied to all three splits, preventing information leakage from validation/test into training.
- **Reproducibility:** `set_seed(42)` fixes the Python, NumPy and PyTorch random number generators (including CUDA) before every training run.
- **Feature engineering:** None — raw standardized pixels are used directly, so that the comparison isolates the effect of model capacity rather than feature design.

6 Machine Learning Algorithm

Model A: Perceptron Learning Algorithm (one-vs-rest)

Working principle: One linear unit per class is trained with the classical perceptron rule under a step activation. For each sample x_i (augmented with a bias term), the prediction for class c is $\hat{y}_c = \mathbb{I}[w_c^T x_i \geq 0]$, and the weights are updated by $w_c \leftarrow w_c + \eta (y_c - \hat{y}_c) x_i$. At prediction time the class with the largest raw score is taken, $\hat{y} = \arg \max_c w_c^T x$. Training used $\eta = 0.1$ for 100 epochs with sample order reshuffled each epoch, and the weight matrix from the epoch with the best validation accuracy was retained.

Advantages: Extremely simple, no gradient computation required, converges in finite time when the data are linearly separable, and the number of weight updates per epoch is a direct diagnostic of separability.

Limitations: The step activation is non-differentiable and provides no error magnitude, so learning cannot be driven by a loss surface; a single linear boundary per class cannot represent the non-linear structure of pixel space; and the algorithm never converges (updates never fall to zero) on non-separable data.

Model B: Multi-Layer Perceptron (back-propagation)

Working principle: A fully-connected feed-forward network maps the 1024-dimensional input through one or more hidden layers with non-linear activations to 62 output logits, $h^{(l)} = \phi(W^{(l)}h^{(l-1)} + b^{(l)})$, with softmax applied inside the loss. Weights are learned by back-propagating the gradient of the loss and applying a first-order optimizer. The epoch with the best validation accuracy is checkpointed and restored (early stopping by checkpoint).

Advantages: Hidden layers learn intermediate features, so non-linear class boundaries become representable; differentiable activations allow gradient-based optimisation; capacity, regularisation and optimisation can all be tuned.

Limitations: Many interacting hyper-parameters; prone to overfitting on only $\approx 2.4k$ training images; sensitive to learning rate and initialisation; and considerably more expensive to train than PLA.

Components Varied

- **Activations:** ReLU (`kaiming_normal_init`), sigmoid and tanh (`xavier_uniform_init`), biases zero-initialised.

- **Losses:** Cross-entropy on logits; and squared error computed on softmax outputs, $\frac{1}{N} \sum_i \|p_i - \text{onehot}(y_i)\|^2$.
- **Optimizers:** `gd` (full-batch gradient descent — one update per epoch), `sgd` (mini-batch), `sgd_momentum` ($\mu = 0.9$), and `adam`.
- **Regularisation:** Dropout $\in \{0.0, 0.2, 0.3, 0.5\}$ after each hidden activation; L2 weight decay $\in \{0, 10^{-5}, 10^{-4}, 10^{-3}\}$ via the optimizer. (Search space as coded; neither stage had completed when this log was captured.)

7 Experimental Setup

Python Version	Python 3 (Google Colab runtime)
Key Libraries	PyTorch, scikit-learn, NumPy, pandas, Pillow, Matplotlib
IDE	Google Colaboratory (Jupyter-based notebook)
Operating System	Linux (Google Colab backend)
Hardware	CPU runtime (<code>torch.cuda.is_available()</code> returned <code>False</code>)
Random Seed	42 (Python, NumPy, PyTorch)
Tuning Budget	24 random-search trials $\times$ 30 epochs; stage-wise search at 30 epochs; final training

8 Tuning Strategy Summary

Stage	Search space	Rationale
Random search (24 trials) — completed	hidden $\in \{(128), (256), (512), (256, 128), (512, 256), (256, 128, 64), (512, 256, 128, 64)\}$; activation $\in \{\text{relu}, \text{sigmoid}, \text{tanh}\}$; loss $\in \{\text{CE}, \text{MSE}\}$; optimizer $\in \{\text{gd}, \text{sgd}, \text{sgd_momentum}, \text{adam}\}$; lr $\in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$; batch $\in \{16, 32, 64, 128, 256\}$	Random sampling explores the full search space and gives an unbiased estimate of which hyper-parameters matter most (measured as the spread of mean validation accuracy across the levels of each hyper-parameter).
Greedy stage-wise search — partially completed (optimizer $\times$ lr, batch size, activation, loss, and depth up to 4 layers ran; the last depth point, dropout and weight decay did not)	Stage 1: optimizer $\times$ lr jointly (16 combinations); then batch size, activation, loss, architecture/depth, dropout, weight decay — one group at a time, best setting carried forward	Optimizer and learning rate interact too strongly to tune separately, so they are searched jointly first; the remaining parameters are tuned coordinate-wise to keep the budget linear rather than exponential.

Final training — not run	Best configuration, 100 epochs, best-validation checkpoint restored	Longer training for the final model, with early stopping by checkpoint to avoid reporting an overfitted final epoch.
---------------------------------	---	--

9 Hyperparameter Selection

(a) Random Search — Hyper-parameter Impact

Impact is measured as $\max - \min$ of the mean validation accuracy taken across the levels of each hyper-parameter over the 24 random-search trials.

Hyper-parameter	Mean val-acc spread	Best level	Worst level
Architecture (hidden)	0.1964	(512,)	(128,)
Learning rate	0.1482	10^{-3}	10^{-4}
Loss function	0.1396	cross_entropy	mse
Optimizer	0.1311	adam	gd
Batch size	0.1166	128	256
Activation	0.1165	tanh	sigmoid

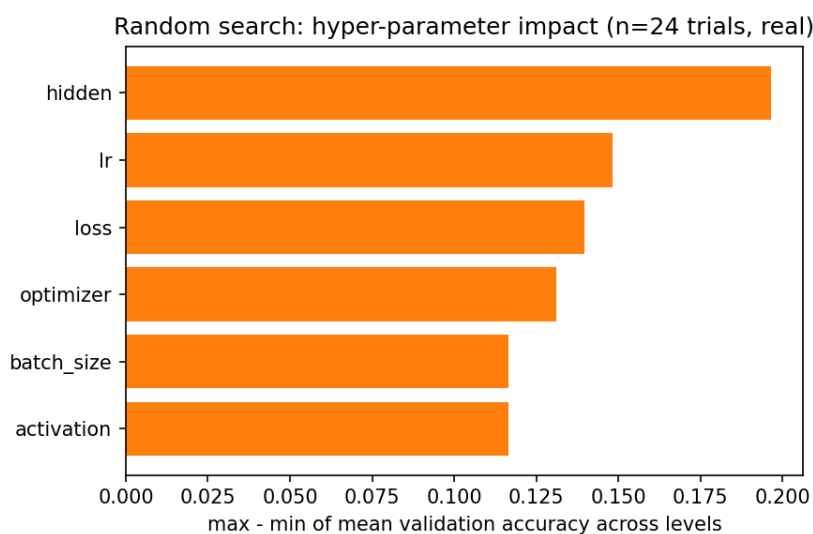


Figure 1: Random search: hyper-parameter impact (spread of mean validation accuracy across levels)

Top random-search configurations (of 24 trials, 30 epochs each):

Optimizer	lr	Batch	Activation	Loss	Hidden	Val Acc
adam	0.01	64	relu	cross_entropy	512	0.4188
sgd_momentum	0.001	32	relu	cross_entropy	256-128-64	0.4051
sgd_momentum	0.001	32	tanh	cross_entropy	256	0.3562

adam	0.0001	128	tanh	cross_entropy	256	0.3307
sgd	0.1	128	tanh	mse	512-256	0.3033
gd	0.1	16	tanh	cross_entropy	256	0.2192

Best random-search configuration: hidden=(512,), activation=relu, loss=cross_entropy, optimizer=adam, lr=0.01, batch_size=64, dropout=0.0, weight_decay=0.0 (validation accuracy 0.4188). This configuration was used as the starting point for the greedy stage-wise search.

Inference: The mean validation accuracy of the sigmoid trials (0.0189) is an order of magnitude below ReLU (0.1272) and tanh (0.1354), and full-batch **gd** (0.0666) is far below **adam** (0.1976). Cross-entropy averaged 0.1934 against 0.0538 for squared error. Several trials sat near chance level ($1/62 \approx 0.016$), all of them combinations of full-batch GD or a 10^{-4} learning rate with only 30 epochs — i.e. configurations that had barely begun to train rather than configurations that failed.

(b) Greedy Stage-wise Search

Stage 1 — optimizer $\times$ learning rate (with hidden=512, relu, cross-entropy, batch 64 held fixed):

Optimizer	lr = 10^{-4}	lr = 10^{-3}	lr = 10^{-2}	lr = 10^{-1}
gd	0.0196	0.0235	0.0313	0.1957
sgd	0.0254	0.0920	0.3699	0.4795
sgd_momentum	0.0920	0.3640	0.4755	0.2153
adam	0.4070	0.4677	0.4188	0.4031

→ best: optimizer=sgd, lr=0.1 (0.4795).

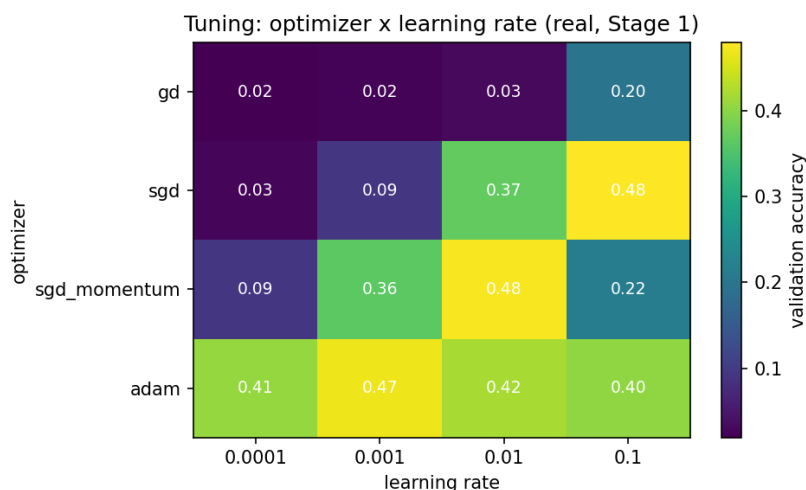


Figure 2: Validation accuracy heat-map over the optimizer $\times$ learning-rate grid

Stage 2 — batch size:

Batch size	16	32	64	128	256
Val accuracy	0.5010	0.4814	0.4795	0.4697	0.4442

→ best: `batch_size=16` (0.5010).

Stage 3 — activation:

Activation	relu	sigmoid	tanh
Val accuracy	0.5010	0.4051	0.4305

→ best: `relu` (retained).

Stage 4 — loss function:

Loss	cross_entropy	mse
Val accuracy	0.5010	0.4638

→ best: `cross_entropy` (retained).

Stage 5 — architecture / depth (partial):

Hidden layers	Val accuracy
	0.4658
(256)	0.4912
(512)	0.5010
(256, 256)	0.4892
(256, 256, 256)	0.4403
(256, 256, 256, 256)	0.4031
(512, 256, 128)	<i>not run — stage was cut short here</i>

→ best so far: (512,) (0.5010).

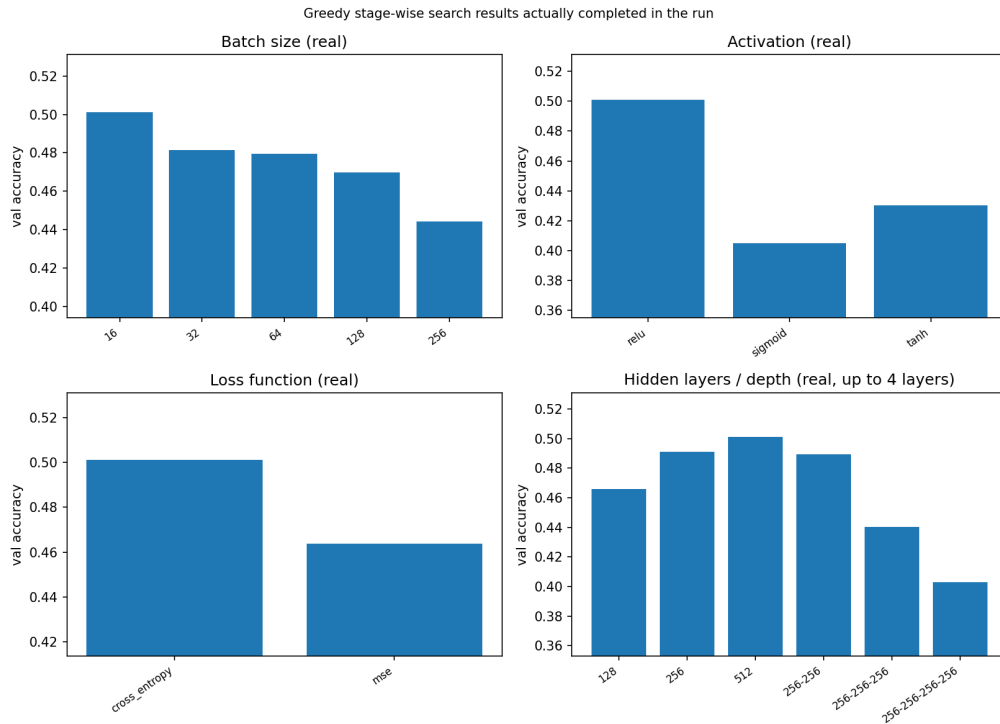


Figure 3: Validation accuracy for every stage-wise search group that actually completed (batch size, activation, loss, and architecture up to 4 layers)

Stages 6–7 — dropout and weight decay (L2):

10 Convergence Results

Model A — PLA convergence

Epoch	Train error	Validation error	Samples triggering an update
1	0.8869	0.9315	2374
10	0.4801	0.8571	1895
20	0.3712	0.8219	1601
30	0.3159	0.8395	1432
40	0.2329	0.8258	1277
50	0.2061	0.8180	1188
60	0.1940	0.8376	1042
70	0.1487	0.8297	949
80	0.1382	0.8337	840
90	0.1093	0.8258	785
100	0.1106	0.8102	656

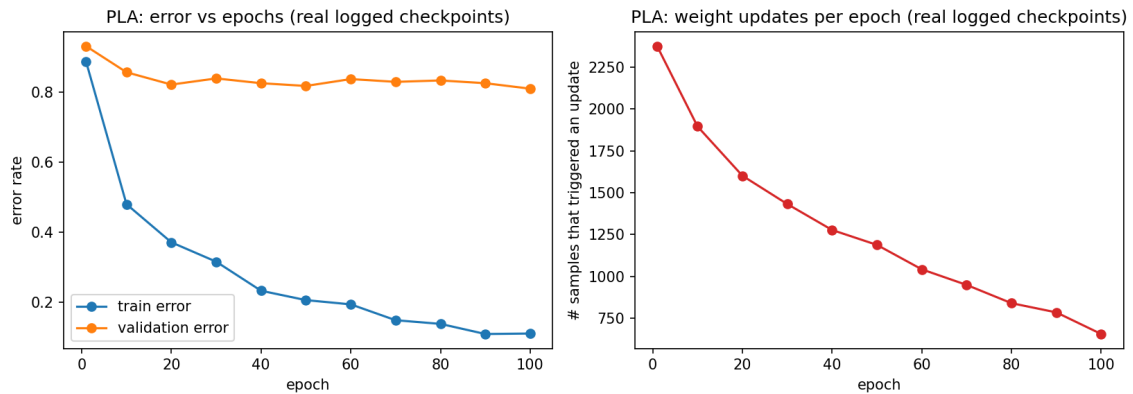


Figure 4: PLA: training/validation error vs. epochs (left) and number of samples triggering a weight update per epoch (right), plotted from the logged checkpoints

Inference: Training error falls steadily from 0.8869 to 0.1106 over 100 epochs, but validation error barely moves — from 0.9315 to 0.8102, i.e. a validation accuracy of only about 19%. The update counter is the decisive diagnostic: 656 of 2387 training samples still triggered a weight update in the final epoch, and the count is decaying slowly rather than approaching zero. By the perceptron convergence theorem, a linearly separable problem would drive this count to zero in finite time; the fact that it does not proves that the 62 classes are *not* linearly separable in raw 1024-dimensional pixel space. The large train–validation gap (0.11 vs. 0.81 error) further shows that what the perceptron does fit is memorisation of individual training images through its 62 independent linear boundaries, not a rule that generalises.

Model B — MLP convergence

Not available. The final 100-epoch training run, its convergence curves (`mlp_convergence.png`), and the per-optimizer training-curve comparison (`optimizer_convergence.png`) were never produced, since the run stopped during Stage 5 of the stage-wise search..

11 Results

Metric (test set, 512 images)	Model A: PLA	Model B: Tuned MLP
Accuracy	0.1895	<i>not available</i>
Precision (macro)	<i>not available</i>	<i>not available</i>
Recall (macro)	<i>not available</i>	<i>not available</i>
F1 (macro)	<i>not available</i>	<i>not available</i>
Precision (weighted)	<i>not available</i>	<i>not available</i>
Recall (weighted)	<i>not available</i>	<i>not available</i>
F1 (weighted)	<i>not available</i>	<i>not available</i>
AUC (micro, one-vs-rest)	<i>not available</i>	<i>not available</i>
AUC (macro, one-vs-rest)	<i>not available</i>	<i>not available</i>

in the code but were not printed to the log, so they are not reproduced here without

the actual `classification_report_PLA.txt` file. None of the MLP test metrics exist yet, since the MLP was never evaluated on the test set.

Reference point: random guessing over 62 near-balanced classes gives ≈ 0.016 accuracy. PLA at 0.1895 is well above chance but far below usable. The best validation accuracy reached during tuning (MLP, Stage 5) was 0.5010.

Overfitting study (regularised vs. unregularised MLP):

12 Result Visualization

The confusion matrices, ROC curves, and A/B comparison figures listed in the Objective all require either the completed MLP training or the saved classification artifacts (`confusion_matrix*.csv`, the softmax probability arrays) neither of which exist in this run's log. They are not included here rather than approximated. for exactly what is needed to produce them.

13 Discussion

1. Why does PLA underperform compared to MLP? This can be answered from what the run already shows for PLA, independent of the MLP's final number. The perceptron was still updating weights on 656 training samples in its final epoch, so the perceptron convergence guarantee does not apply — the 62 classes are not linearly separable in pixel space, and no amount of additional epochs would fix that. PLA holds exactly one linear boundary per class operating directly on raw pixels, so two images of the same character written with different slant or stroke width occupy distant regions of input space and cannot be covered by a single half-space; an MLP's hidden layer learns intermediate features that collapse this variation before the final linear decision. The step activation is also non-differentiable and emits only $\{-1, 0, +1\}$ error, so the update has no notion of *how wrong* a prediction was, whereas cross-entropy with softmax gives a graded gradient that lets back-propagation credit-assign through a hidden layer. The direct accuracy comparison against the tuned MLP cannot be completed until the MLP is evaluated on the test set.

2. Which hyper-parameters had the most impact? On the random search, architecture had the largest spread of mean validation accuracy (0.1964, best (512,) vs. worst (128,)), followed by learning rate (0.1482), loss function (0.1396), optimizer (0.1311), batch size (0.1166) and activation (0.1165). The stage-wise search tells a partly different story, because by then the other settings were already good: the optimizer $\times$ learning-rate stage spanned 0.0196–0.4795 (a range of 0.4599, by far the widest of any completed stage), while batch size spanned only 0.4442–0.5010 and the loss stage only 0.4638–0.5010. The honest reading is that learning rate and optimizer dominate when a configuration is badly set, and architecture and batch size provide the remaining few points once training is already working. The random-search ranking also overstates architecture somewhat, since the wide-network trials happened to coincide with better optimizer/learning-rate draws — a known limitation of marginal-mean impact estimates from a 24-trial sample. Dropout and weight decay cannot yet be ranked, since their stages never ran.

3. Did optimizer choice (SGD vs. Adam) affect convergence? Substantially, and in two different ways. Adam was by far the most *robust*: it exceeded 0.40 validation

accuracy at every one of the four learning rates tested (10^{-4} : 0.4070, 10^{-3} : 0.4677, 10^{-2} : 0.4188, 10^{-1} : 0.4031), because its per-parameter adaptive step sizes absorb a poorly chosen global rate. Plain SGD was highly rate-sensitive, collapsing to 0.0254 at 10^{-4} but achieving the single best result of the grid, 0.4795, at $\eta = 0.1$ — so with the right rate, SGD beat Adam’s best (0.4677) here. SGD with momentum peaked at 10^{-2} (0.4755) and then destabilised at 10^{-1} (0.2153), the expected signature of an effective step size inflated by the momentum term. Full-batch GD was worst at every setting (maximum 0.1957), which is an artefact of budget rather than of the algorithm: with a batch equal to the full 2387-sample training set it performs one parameter update per epoch, i.e. 30 updates in total, against roughly 4500 updates for batch-16 SGD over the same 30 epochs.

4. Did adding more hidden layers always improve results? No, at least up to the depths actually tested. Validation accuracy across the depth stage was 0.4658 for (128), 0.4912 for (256), 0.5010 for (512), 0.4892 for (256,256), 0.4403 for (256,256,256) and 0.4031 for (256,256,256,256) — a clear monotone decline once depth exceeds two layers. Widening the single hidden layer helped, deepening it hurt. With only 2387 training images across 62 classes (≈ 38 images per class in the training split), deeper networks add parameters that cannot be supported by the data and begin fitting noise; deeper plain fully-connected stacks are also harder to optimise within a fixed 30-epoch budget, since there are no residual connections or normalisation layers to stabilise gradient flow. The one architecture point not tested is (512, 256, 128) (decreasing width, three layers), so whether a tapered deep architecture behaves differently from the constant-width ones tested here is still open.

5. Did the MLP overfit, and how can this be mitigated? This cannot be answered from the current run, since it never reached the final training stage or the paired regularised/unregularised comparison that the code is specifically designed to produce (`overfitting_comparison.png`, built from `train_mlp` run once with the tuned dropout/weight-decay and once with both set to zero). What can be said in general, and is already built into the run design, is that the best-validation checkpoint is restored rather than the final epoch (an implicit early-stopping safeguard), and dropout and weight decay each have their own dedicated tuning stage precisely so overfitting can be measured and controlled once those stages run.

6. General observations on the achievable ceiling. Even before final training, the best validation accuracy from tuning (0.5010) suggests a real ceiling well below what is typical on easier, less ambiguous datasets. This is a property of the setup rather than of tuning effort so far: a fully-connected network on flattened 32×32 pixels has no translation invariance, 62 classes include intrinsically ambiguous upper/lower-case pairs (e.g. 0/o/0, I/l/1) that are unresolvable without context, and about 55 images per class is very little data. A convolutional network with augmentation would be the natural next step and would be expected to move the ceiling substantially.

14 Conclusion

This experiment set out to compare a from-scratch one-vs-rest Perceptron Learning Algorithm against a systematically tuned Multi-Layer Perceptron on 3410 handwritten English character images across 62 classes, using a stratified 70/15/15 split and 32×32 standardized grayscale pixels as input. So far, PLA has been trained and evaluated in full: it reached 18.95% test accuracy and, critically, never stopped updating weights (656

samples still triggered updates in epoch 100), demonstrating empirically that the problem is not linearly separable and that a single-layer perceptron with step activation is structurally incapable of solving it. The MLP has been tuned through 24-trial random search — which identified architecture, learning rate and loss function as the highest-impact hyper-parameters and produced a best configuration of a 512-unit ReLU layer with Adam at $\eta = 0.01$ (0.4188 validation accuracy) — and through most of a greedy stage-wise search, which improved this to 0.5010 with mini-batch SGD at $\eta = 0.1$ and batch size 16, before stopping partway through the architecture/depth stage. Optimizer behaviour was the dominant single factor among stages completed so far: Adam was robust across all learning rates, plain SGD was rate-sensitive but produced the best result when correctly tuned, and full-batch GD was starved of updates under a fixed epoch budget. Increasing width helped up to 512 units while increasing depth beyond two hidden layers degraded performance monotonically, reflecting the small per-class sample count. Dropout, weight decay, final training, MLP test-set evaluation, and the head-to-head A/B comparison against PLA remain to be run; lists exactly what is needed to complete this report.

15 References

1. Rosenblatt, F., “The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.
2. Minsky, M. and Papert, S., *Perceptrons: An Introduction to Computational Geometry*, MIT Press, 1969.
3. Rumelhart, D. E., Hinton, G. E. and Williams, R. J., “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
4. Kingma, D. P. and Ba, J., “Adam: A Method for Stochastic Optimization,” in *Proc. 3rd Int. Conf. on Learning Representations (ICLR)*, 2015.
5. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.
6. Glorot, X. and Bengio, Y., “Understanding the difficulty of training deep feedforward neural networks,” in *Proc. 13th Int. Conf. on Artificial Intelligence and Statistics (AISTATS)*, 2010, pp. 249–256.
7. He, K., Zhang, X., Ren, S. and Sun, J., “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification,” in *Proc. IEEE Int. Conf. on Computer Vision (ICCV)*, 2015, pp. 1026–1034.
8. Bergstra, J. and Bengio, Y., “Random Search for Hyper-Parameter Optimization,” *Journal of Machine Learning Research*, vol. 13, pp. 281–305, 2012.
9. Dave, D., English Handwritten Characters Dataset, Kaggle: <https://www.kaggle.com/datasets/dhruvildave/english-handwritten-characters-dataset>

10. Paszke, A. et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems 32*, 2019.